

SBI EngageAI

Autonomous AI Financial Companion — Complete Product, Architecture & Build Specification

SBI Agentic AI Hackathon — Theme: Digital Engagement

Version 1.0

Table of Contents

- Document Purpose
 - 1. Product Vision
 - 1.1 What we are building
 - 1.2 Why this wins on the “Digital Engagement” theme
 - 2. Product Naming
 - 3. Core Concept
 - 3.1 The shift: from reactive to proactive
 - 3.2 The customer experience promise
 - 4. Customer Data Model — The Financial Life Graph
 - 4.1 Concept
 - 4.2 Schema (logical model)
 - 4.3 How the graph evolves
 - 5. Life Event Detection Engine
 - 5.1 Purpose
 - 5.2 Detection logic, signals, and examples
 - 5.3 Confidence scoring
 - 5.4 Detection technique
 - 6. Agentic AI Architecture
 - 6.1 Why multi-agent, not a single LLM call
 - 6.2 The six agents
 - 6.3 Full workflow example (end-to-end)
 - 6.4 Orchestration pattern
 - 6.5 Read/write separation
 - 7. Digital Engagement — Scenario Playbook
 - 8. Digital Adoption — As an Outcome, Not a Module
 - 8.1 The philosophy
 - 8.2 Why this matters for the score
 - 9. Customer Acquisition — Emerging Naturally from the Same Engine
 - 9.1 The acquisition surface
 - 9.2 How trust converts to acquisition
 - 10. Demo Strategy — Synthetic Customer Generator
 - 10.1 Why synthetic data, done properly, is a strength not a workaround
 - 10.2 Generation method
 - 10.3 The six seed personas
 - 10.4 What the generator outputs per persona
 - 11. Judge Experience — Demo User Journey
 - 12. Dashboard Design — UX Specification
 - 12.1 Landing Page
 - 12.2 Customer Generator
 - 12.3 Customer Dashboard
 - 12.4 Financial Life Graph View
 - 12.5 Agent Activity Center
 - 12.6 AI Chat Interface
 - 12.7 Recommendations Center
 - 12.8 Action Center
 - 13. Agent Reasoning Visualization
 - 13.1 Concept
 - 13.2 Implementation
 - 13.3 Visual treatment
 - 14. Technical Architecture
 - 14.1 Stack
 - 14.2 High-level architecture diagram (text form)
 - 14.3 Component interactions
 - 14.4 Core API surface
 - 14.5 Data flow (one engagement cycle)
 - 14.6 Database schema (core tables)
 - 15. Hackathon MVP Scope
 - 15.1 Must Have (build first, demo-blocking)
 - 15.2 Should Have (strong differentiation if time allows)
 - 15.3 Nice to Have (only if ahead of schedule)

15.4 What is realistic in 24–48 hours

16. Business Impact & KPIs

16.1 How SBI benefits

17. Commercial Potential

17.1 Revenue impact

17.2 Cross-selling impact

17.3 Customer retention impact

17.4 Operational cost reduction

18. Round 1 Submission Content

19. Pitch Deck — 12 Slides

20. Demo Script (3 Minutes)

21. Differentiators

22. Build Checklist for an Autonomous Coding Agent

Document Purpose

This document is a complete, implementation-ready specification for an autonomous AI financial companion built for the SBI Agentic AI Hackathon under the **Digital Engagement** theme. It is written to be machine-readable and directly actionable: an AI coding agent (e.g. Antigravity AI) or a human engineering team should be able to read this file top to bottom and begin scaffolding the repository, data models, agent graph, APIs, and UI without needing additional clarification.

It contains, in order: product vision and naming, the core conceptual model, the customer data model (Financial Life Graph), the Life Event Detection Engine, the six-agent agentic architecture with full workflows, the Digital Engagement scenario playbook, the Digital Adoption and Customer Acquisition philosophy, the synthetic data strategy, the judge demo journey, the UX specification for eight product surfaces, the agent reasoning visualization system, the production technical architecture (with diagrams, schemas, and API contracts), a 24-48 hour MVP build plan, business impact and commercial modeling, hackathon Round 1 submission copy, a 12-slide pitch deck, a 3-minute demo script, competitive differentiation, and a final build checklist for an autonomous coding agent.

1. Product Vision

1.1 What we are building

We are **not** building a banking app. We are **not** building another chatbot. We are building an **AI layer that sits on top of SBI's existing banking ecosystem** — an autonomous agent network that continuously observes a customer's financial life, reasons about what is happening in it, and proactively engages the customer at the right moment with the right action.

Traditional digital banking is reactive: the customer opens the app, searches for what they need, and the bank responds. Our system inverts this relationship. The AI:

1. **Understands** customer behavior continuously, not at login time.
2. **Analyzes** transaction patterns to build a living picture of financial health.
3. **Detects** life events as they emerge from data, before the customer mentions them.
4. **Predicts** future financial needs based on life-stage trajectory.
5. **Engages** the customer proactively, through the channel and moment most likely to land.
6. **Recommends** SBI products that are genuinely relevant, not generically pushed.
7. **Builds financial plans** that adapt as circumstances change.
8. **Follows up over time**, the way a good relationship manager would, instead of dropping a one-time notification.
9. **Drives product adoption** as a natural by-product of being useful, not as a sales funnel.

The mental model is best captured as: **“ChatGPT + Financial Advisor + Autonomous Banking Agent.”** It has the conversational fluency of a modern LLM assistant, the judgment of a human financial advisor, and the persistence and tool-use of an autonomous agent that takes real (or in the demo, simulated) actions on the customer's behalf.

1.2 Why this wins on the “Digital Engagement” theme

The problem statement asks for AI-driven engagement models that proactively interact with customers based on **behaviors, financial patterns, and life events**. Most teams will build a recommendation chatbot — a system that waits for a question and then answers well. That satisfies “AI” but not “proactive engagement.” Our differentiator is that the system **initiates** the interaction. The customer never has to ask; the AI already noticed the salary credit, the new EMI, the spike in airline spending, or the school-fee debit, and has already reasoned about what it means before the customer opens the app.

Digital Adoption and Customer Acquisition (the two secondary themes the brief permits) are not bolted on as separate modules. They are **outcomes of engagement**, addressed in Sections 7 and 8.

2. Product Naming

Below are five candidate names, evaluated against three criteria: clarity of value proposition, fit with SBI’s brand voice, and memorability for judges.

Name	Read on first hearing	Strength	Weakness
SBI EngageAI	Descriptive, on-theme	Directly names the hackathon theme; unambiguous for judges scoring against the problem statement	Slightly generic as a long-term consumer brand
SBI LifeOS	Bold, platform-scale	Strong “operating system for your financial life” metaphor; great for the data-graph narrative	Less obviously about engagement; sounds infrastructural
SBI Companion AI	Warm, relationship-first	Reinforces the “relationship manager” feel	“Companion AI” is a crowded category name (consumer companion apps)
SBI Financial Copilot	Familiar (Copilot pattern), credible	Instantly understood metaphor (assists, doesn’t replace)	Slightly derivative of Microsoft Copilot branding
SBI Life Agent	Short, agentic	Emphasizes autonomy directly	Could be confused with insurance “agent” terminology in banking context

Recommendation: **SBI EngageAI**, with “**Financial Copilot**” retained as the in-product tagline (“*SBI EngageAI — Your Financial Copilot*”). This gives the submission a name that scores directly against the stated theme while keeping a secondary phrase that communicates the product metaphor to judges and end users in one sentence. All further sections use **SBI EngageAI** as the product name.

3. Core Concept

3.1 The shift: from reactive to proactive

	Traditional Banking	SBI EngageAI
Trigger	Customer initiates	System initiates
Timing	Whenever customer logs in	Moment of relevance (event-driven)
Personalization	Segment-level (one offer to one customer segment)	Individual-level (reasoned from that customer's own graph)
Channel logic	Static (push notification to everyone)	Adaptive (channel and tone chosen by engagement strategy agent)
Memory	Stateless per session	Persistent — remembers prior conversations, declined offers, and follow-up state
Output	A product page	A reasoned recommendation with explanation and a concrete next action

The core operating loop is:

```
AI detects → AI reasons → AI engages → AI recommends → AI follows up
```

This loop runs continuously and asynchronously in the backend (via scheduled and event-triggered agent runs), independent of whether the customer has the app open. When the customer does open the app, or when a high-confidence opportunity is found, the loop surfaces a touchpoint — an in-app card, a chat-initiated message, a push notification, or (in the synthetic demo) a step in the Agent Activity Center.

3.2 The customer experience promise

The product's single experiential promise is: **the customer should feel like SBI understands their financial journey** — not their transaction list, their *journey*. Concretely this means every recommendation the system makes is traceable to a chain of reasoning the customer (and the judge) can inspect, in plain language, in under ten seconds. This traceability is also the centerpiece demo feature (Section 12, Agent Reasoning Visualization).

4. Customer Data Model — The Financial Life Graph

4.1 Concept

The **Financial Life Graph (FLG)** is the single, continuously-evolving representation of a customer that all six agents read from and write to. It is not a static KYC record; it is a graph that accumulates inferred state over time, where every node carries a confidence score and a “last observed” timestamp, and every inference is linked back to the evidence (transactions, declared data, or detected events) that produced it. Treating it as a *graph* rather than a flat profile matters because financial life is relational: a “child birth” event connects to education-goal nodes, insurance nodes, and savings-capacity nodes simultaneously, and the agents need to traverse those relationships, not just read fields.

4.2 Schema (logical model)

```
{
  "customer_id": "string",
  "demographics": {
    "age": "int",
    "occupation": "string",
    "city_tier": "1|2|3",
    "marital_status": "string",
    "dependents": "int",
    "life_stage": "student|young_professional|new_parent|home_buyer|established|pre_retirement|retiree"
  },
  "income": {
    "primary_monthly": "float",
    "income_type": "salaried|self-employed|business",
    "income_trend_90d": "rising|stable|falling",
    "last_salary_credit": "date",
    "income_volatility_score": "0-1"
  },
  "expenses": {
    "fixed_monthly": "float",
    "discretionary_monthly": "float",
    "category_breakdown": {
      "rent": "float", "groceries": "float", "travel": "float", "education": "float", "healthcare": "float", "entertainment": "float", "emi": "float"
    },
    "expense_trend_90d": "rising|stable|falling"
  },
  "savings": {
    "liquid_balance": "float",
    "avg_monthly_surplus": "float",
    "idle_funds_score": "0-1",
    "savings_rate_pct": "float"
  },
  "investments": {
    "instruments_held": ["FD", "RD", "SIP", "mutual_fund", "PPF", "NPS", "stocks"],
    "total_aum": "float",
    "sip_active": "bool",
    "risk_profile": "conservative|moderate|aggressive"
  },
  "loans": {
    "active_loans": [{"type": "home|auto|personal|education", "outstanding": "float", "emi": "float", "tenure_remaining_months": "int"}],
    "credit_utilization_pct": "float",
    "loan_eligibility_headroom": "float"
  },
  "insurance": {
    "policies_held": ["term", "health", "motor", "travel"],
    "coverage_gap_score": "0-1"
  },
  "financial_goals": {
    "goal_type": "home|education|retirement|travel|wedding", "target_amount": "float", "target_date": "date", "funded_pct": "float", "confidence": "declared|inferred"
  },
  "product_usage": {
    "sbi_products_held": ["string"],
    "digital_channels_active": ["yono", "net_banking", "upi"],
    "engagement_score": "0-1"
  },
  "predicted_future_needs": {
    "need": "string", "probability": "0-1", "horizon_days": "int", "evidence_refs": ["transaction_id", "event_id"]
  },
  "graph_metadata": {
    "last_updated": "timestamp",
    "confidence_overall": "0-1",
    "node_sources": "audit trail mapping each field to its originating signal"
  }
}
```

4.3 How the graph evolves

Every transaction, every declared field update (e.g. KYC refresh), and every detected life event is an **edge-writing event**. The Financial Behavior Analyzer Agent (Section 5) is the primary writer for behavioral nodes; the Life Event Detection Agent writes life-stage and event

nodes; the Opportunity Discovery Agent reads the graph but does not write to it (kept read-only to prevent circular reasoning). This separation of read/write responsibility per agent is itself a key architecture decision documented in Section 5.6.

Concretely, in the demo build, the Financial Life Graph is a row-per-customer JSON document in PostgreSQL (`customer_graph` table, `JSONB` column) with a parallel `graph_events` audit table recording every write, who (which agent) made it, and what evidence triggered it. This audit table is what powers the Agent Reasoning Visualization in Section 12 — the visualization is literally a rendering of the audit trail, not a separately authored narrative.

5. Life Event Detection Engine

5.1 Purpose

The Life Event Detection Engine converts raw transaction and behavioral signals into discrete, named life events with a confidence score. This is the layer that makes the rest of the system feel intelligent rather than rule-based, because life events are rarely announced by a single transaction — they are inferred from a *pattern* of signals over a window of time, the same way a human relationship manager would notice “something’s changed” from looking at a statement.

5.2 Detection logic, signals, and examples

Life Event	Primary Signals	Transaction Pattern Window	AI Reasoning Example
New Job	New salary-credit narration/payer, old payer stops	60 days	“Salary credits from Payer A stopped after 1 Mar; salary credits from Payer B began 5 Mar at a similar date-of-month — treat as employer change, not income loss.”
Salary Hike	Same payer, monthly credit amount increases >12% and holds for 2+ cycles	90 days	“Same employer narration, credit amount rose from ₹62,000 to ₹74,000 and has now repeated twice — increase confidence above the single-month noise threshold.”
Marriage	Joint-account opening, surname-pattern change in linked KYC, spike in jewellery/wedding-vendor category spend, new beneficiary added matching a new linked relationship	120 days	“Large one-time spends across catering, jewellery and venue-category merchants within a 30-day cluster, followed by a new joint-mode account request — high-confidence marriage event.”
Child Birth	New hospital/maternity merchant spend, new recurring pediatric/pharmacy spend, insurance nominee-addition request	90 days	“Maternity-hospital debit followed within 45 days by recurring pediatric-category debits — strong signal for a new dependent.”
Relocation	Change in ATM/UPI geotag cluster, address-update request, change in recurring rent-narration payee	60 days	“ATM and UPI merchant geotags shifted from Pune cluster to Bengaluru cluster and have stayed there for 6 consecutive weeks — relocation, not travel.”
Frequent Travel	Repeated airline/rail/OTA merchant categories, multiple city geotags, no sustained relocation pattern	90 days rolling	“Six distinct airline-category transactions across four cities in 90 days with geotags reverting to the home cluster each time — frequent traveller, not relocation.”
Vehicle Purchase	Large one-time auto-dealer debit, new auto-loan disbursement, fuel-category spend begins	30 days	“One-time large debit to an auto-dealer merchant code followed by a new auto-loan disbursement credit of matching tenor — vehicle purchase confirmed.”
Home Purchase Intent	Repeated property-portal browsing-linked UPI test payments, builder/broker-category transactions, savings-rate increase with stated goal	180 days	“Savings rate rose 18% over two quarters while small token payments to real-estate brokerage merchant codes appear twice — intent signal, not yet a purchase.”
Education Planning	Recurring tuition/coaching-category debits, education-loan enquiry, age of dependent crossing school-entry threshold	180 days	“Dependent recorded in KYC turning 5 this quarter, combined with two new coaching-institute debits — education planning window opening.”
Retirement Planning	Age crossing 50, declining income-trend flag absent (still working) but increased NPS/PPF contribution, reduced loan EMI burden	365 days	“Customer crossed age 52 with no income disruption, NPS contribution increased 20%, and outstanding loan balance fell below 10% of past peak — retirement-planning life stage.”

5.3 Confidence scoring

Each detected event carries a confidence score (0–1) computed from: (a) signal strength (how far the metric deviates from the customer’s own baseline, not a population baseline), (b) signal persistence (how many consecutive windows confirm it), and (c) corroboration (how many independent signal types agree). Events below 0.55 confidence are logged but not surfaced to the customer; events 0.55–0.8 are surfaced

softly (a suggestion, easily dismissed); events above 0.8 trigger the full engagement workflow. This threshold structure is what prevents the system from feeling like a noisy, over-eager bot — a frequent failure mode of naive “AI banking” demos.

5.4 Detection technique

In production this would be a hybrid of (1) deterministic feature engineering over the transaction stream (rolling averages, change-point detection, merchant-category-code clustering, geotag clustering) producing structured features, and (2) an LLM-based reasoning pass that takes the structured features plus a short natural-language transaction summary and classifies/explains the event, because pure rule-based systems cannot weigh corroborating-but-imperfect signals the way the examples above require. For the hackathon build, the Synthetic Customer Generator (Section 9) seeds personas with pre-labelled life events, and the detection engine is demonstrated by replaying that persona’s transaction stream through the same feature-engineering + LLM-reasoning pipeline that would run against real data — this is the key “we built the real thing, not a mock” proof point for judges.

6. Agentic AI Architecture

6.1 Why multi-agent, not a single LLM call

A single prompt that tries to “analyze the customer and recommend something” collapses distinct responsibilities (pattern detection, event classification, opportunity scoring, message crafting, action planning, and follow-up scheduling) into one opaque step, which is both harder to make reliable and impossible to visualize for the judge. Splitting these into six specialized agents with explicit inputs/outputs gives us (a) better reliability per step, (b) a natural narrative for the reasoning visualization, and (c) a system that can be extended one agent at a time after the hackathon.

6.2 The six agents

Agent 1 — Financial Behavior Analyzer Agent

- **Purpose:** Continuously summarize a customer’s transaction stream into structured behavioral features and update the relevant nodes of the Financial Life Graph.
- **Inputs:** Raw/synthetic transaction stream, existing Financial Life Graph snapshot.
- **Outputs:** Updated behavior nodes (income trend, expense trend, savings rate, idle-funds score, category breakdown), a list of “notable changes” since the last run.
- **Reasoning process:** Computes rolling statistical features over configurable windows (30/60/90/180 days), compares each against the customer’s own historical baseline (not a population average, to avoid bias against atypical-but-healthy customers), and emits only the deltas that cross a materiality threshold.
- **Interactions:** Writes to the Financial Life Graph; its “notable changes” output is the primary input to Agent 2.

Agent 2 — Life Event Detection Agent

- **Purpose:** Convert notable behavioral changes into named, confidence-scored life events (Section 5).
- **Inputs:** “Notable changes” from Agent 1, current Financial Life Graph (for corroboration, e.g. age, dependents).
- **Outputs:** A list of `{event_type, confidence, evidence_refs, reasoning_text}` objects.
- **Reasoning process:** Pattern-matches notable changes against the event signature table (Section 5.2), then uses an LLM reasoning pass to weigh corroboration and produce the human-readable `reasoning_text` that will later be shown verbatim in the Agent Reasoning Visualization.
- **Interactions:** Writes life-stage/event nodes to the Financial Life Graph; passes high-confidence events to Agent 3.

Agent 3 — Opportunity Discovery Agent

- **Purpose:** Map detected life events and behavioral state to a ranked list of relevant SBI products/actions.
- **Inputs:** Life events from Agent 2, full Financial Life Graph (read-only), a catalog of SBI products with eligibility rules and typical-fit profiles.
- **Outputs:** A ranked opportunity list: `{product, fit_score, rationale, urgency}`.
- **Reasoning process:** For each candidate product, checks hard eligibility (e.g. income floor, existing product overlap) then scores soft fit using the life event + graph context (e.g. “child birth + no existing child-education goal node + healthy savings rate” → high fit for a Recurring Deposit/Child Education Plan combination). Deliberately does **not** write to the graph, to keep opportunity scoring a pure function of current state — this avoids feedback loops where a rejected suggestion silently biases future scoring.
- **Interactions:** Output feeds Agent 4.

Agent 4 — Engagement Strategy Agent

- **Purpose:** Decide *how, when, and through which channel* to present an opportunity — the layer responsible for the product not feeling like spam.
- **Inputs:** Ranked opportunities from Agent 3, customer’s engagement history (channel responsiveness, past dismissals, preferred contact times), `product_usage` node from the graph.
- **Outputs:** An engagement plan: `{opportunity, channel (chat/push/in-app card/email), tone (informational/urgent/celebratory), timing, message_draft}`.
- **Reasoning process:** Applies a frequency cap (never more than N proactive touches per week), avoids re-pitching a recently-dismissed opportunity, and selects tone based on event type (celebratory for marriage/child birth, advisory for idle-funds, time-sensitive for a loan-rate window).
- **Interactions:** Output feeds Agent 5 and is what the customer actually sees first.

Agent 5 — Action Planning Agent

- **Purpose:** Turn an accepted or in-progress opportunity into a concrete, executable plan with steps the customer (or system, with consent) can carry out.
- **Inputs:** Engagement plan from Agent 4 plus customer response (accepted/asked a question/deferred).
- **Outputs:** A step-by-step action plan, e.g. for “increase SIP by ₹5,000”: `{step 1: confirm amount, step 2: select fund, step 3: mock-execute via Action Center, step 4: schedule confirmation}`.
- **Reasoning process:** Decomposes the chosen product/action into the minimum sequence of steps needed for execution, pulling any required eligibility or KYC checks from the graph so the customer isn’t asked for data the bank already has.
- **Interactions:** Writes the resulting plan/action status back to `product_usage` in the Financial Life Graph; hands off to Agent 6 for

Agent 6 — Follow-Up Agent

- ### 6.3 Full workflow example (end-to-end)



6.5 Read/write separation

A deliberate architectural rule: **Agents 1, 2, and 5 write to the Financial Life Graph; Agents 3, 4, and 6 are read-mostly** (Agent 6 writes only follow-up scheduling state, not core graph fields). This prevents opportunity scoring or engagement-strategy decisions from silently mutating the “ground truth” customer state, which would make the audit trail (and therefore the reasoning visualization and any future compliance review) unreliable. Every write carries the writing agent’s ID and the evidence reference, satisfying both explainability and, in a real deployment, RBI-aligned model-governance expectations around explainable credit/engagement decisions.

7. Digital Engagement — Scenario Playbook

The primary objective of the product is engagement, so this section enumerates the scenario library the Opportunity Discovery and Engagement Strategy agents draw from. Each row is a reusable pattern: `{trigger signal} → {AI reasoning} → {engagement action}`.

#	Trigger Signal	AI Reasoning	Engagement Action
1	Salary increase ≥12% sustained 2 cycles	Higher surplus capacity, no current SIP top-up	Recommend increasing SIP amount
2	Frequent travel pattern detected	Multiple trips, no travel insurance held	Recommend travel insurance before next detected trip window
3	Rising education-category spend + dependent age signal	School-entry age approaching, no education goal funded	Suggest Education Planning RD / Sukanya-type product where applicable
4	Idle liquid balance above 3x average monthly expense for 60+ days	Capital sitting non-productively	Recommend liquid fund / short-term FD laddering
5	New job detected (employer change, similar pay)	Possible break in EPF continuity	Suggest EPF transfer / continuity check-in
6	Salary hike + no existing term insurance	Income protection gap widening as income rises	Recommend term life cover sized to new income
7	Marriage event detected	Household financial merging likely	Suggest joint account / household budgeting goal setup
8	Child birth detected	New dependent, no nominee update	Prompt nominee addition across existing policies/accounts
9	Vehicle purchase detected, no motor insurance linked in-app	Compliance + protection gap	Surface motor insurance renewal reminders pre-emptively
10	Home purchase intent signals	Long planning horizon	Offer a home-loan eligibility pre-check, no application pressure
11	Credit utilization climbing toward limit	Risk of revolving high-cost debt	Suggest a personal loan top-up at lower rate to consolidate, or a credit-limit review
12	Declining discretionary spend + rising savings rate over a quarter	Possible belt-tightening or goal-focus	Check in conversationally rather than upsell — protect trust
13	Repeated international merchant spend	Possible upcoming travel or NRI-adjacent need	Suggest forex card / preferential FX rate product
14	Retirement-stage life signals (age + reduced loan burden)	Shift from accumulation to preservation	Suggest NPS/PPF rebalancing conversation
15	Dormant SIP (no increase in 18+ months despite rising income)	Under-indexed investing relative to capacity	Gentle “your SIP hasn’t kept pace with your income” nudge
16	First salary credit ever detected (new-to-bank young professional)	Onboarding moment	Bundle: zero-balance savings perks, starter SIP, basic term cover explainer
17	Large one-time inflow (bonus, maturity payout)	Lump-sum deployment decision point	Present a 3-option allocation plan (debt repayment / investment / emergency fund top-up)
18	Frequent small personal loans / BNPL-style merchant debits	Possible cash-flow stress	Offer a consolidated, lower-cost credit line instead of repeated micro-borrowing
19	Health-category spend spike, no health insurance	Out-of-pocket exposure	Recommend health insurance top-up sized to recent spend pattern
20	Goal funded_pct crosses 90% for a stated goal	Goal nearly complete	Proactively suggest the next goal (e.g. home goal completing → suggest furnishing/relocation fund)

This library is intentionally designed as **data**, not hard-coded logic — in the technical build (Section 13) it lives as a seedable table the Opportunity Discovery Agent scores against, so adding a 21st scenario is a data change, not a redeploy.

8. Digital Adoption — As an Outcome, Not a Module

8.1 The philosophy

A common hackathon mistake is to build “Digital Engagement” and “Digital Adoption” as two separate tabs: an engagement feed, and a products catalog. This treats adoption as something the customer must independently go discover, which reintroduces the reactive pattern the entire product exists to eliminate. In SBI EngageAI, **there is no separate products catalog screen for the customer to browse**. Mutual funds, insurance, credit cards, loans, and even mobile-banking feature adoption (e.g. turning on UPI autopay, activating a savings goal tracker) only ever appear *as the resolution of an engagement* — the direct output of Agent 4 and Agent 5 in Section 6. The customer doesn’t shop; the right product surfaces at the moment its relevance was just established by reasoning the customer can see.

8.2 Why this matters for the score

Judges scoring “Digital Adoption” as a secondary theme will look for adoption funnels and conversion mechanics. We satisfy this not with a separate adoption module but by showing, in the demo, the **adoption rate generated per engagement scenario** (Section 15 KPIs) — i.e., proving the loop converts, rather than building a second product to prove we thought about adoption. This is a stronger answer to the brief’s instruction that adoption “should not be a separate module” and should instead be explained as a philosophy, which is exactly what this section does.

9. Customer Acquisition — Emerging Naturally from the Same Engine

9.1 The acquisition surface

The same six-agent engine, when exposed through a public (non-authenticated, non-account-linked) surface, becomes a **financial guidance entry point** for prospects who are not yet SBI customers. A visitor can describe their situation in chat (“I just got my first job, what should I do with my salary”), and the Opportunity Discovery and Engagement Strategy agents run against a **declared-data-only** version of the Financial Life Graph (no transaction history, just what the user tells the assistant), producing the same quality of reasoned guidance.

9.2 How trust converts to acquisition

1. **Guidance without a pitch:** The first several interactions are pure financial guidance — budgeting logic, goal-setting, product-agnostic explanations — establishing the same “this actually understands me” feeling that drives engagement for existing customers.
2. **Natural product surfacing:** Only once a genuine fit emerges (e.g. the prospect describes a clear savings goal with no current vehicle for it) does the assistant suggest an SBI product, using the identical Opportunity Discovery logic — there is no separate “acquisition mode” prompt engineering, which keeps the experience consistent and the recommendation credible.
3. **Low-friction conversion step:** The suggested next action is always the lightest possible (e.g. “open a zero-balance digital savings account in 3 minutes”) rather than a full product application, converting trust into an account relationship first, and a fuller product relationship over time through the same continuous engagement loop the rest of this document describes.

This means **acquisition and engagement run on one codebase and one reasoning engine** — the only difference is which data source feeds the Financial Life Graph (live banking data vs. self-declared chat data) — which is the cleanest possible answer to “show how the same engagement engine can be exposed publicly.”

10. Demo Strategy — Synthetic Customer Generator

10.1 Why synthetic data, done properly, is a strength not a workaround

We have no access to real SBI customer data, so the demo must be built on synthetic personas. Done poorly, this looks like a hard-coded demo. Done properly — generated by the same AI techniques that would later process real data, with realistic statistical variance and embedded life-event ground truth — it becomes a *feature*: a working Synthetic Customer Generator is itself reusable for SBI’s internal model testing, QA, and training-data needs after the hackathon, which is a credible commercial talking point (Section 16).

10.2 Generation method

An LLM-driven generator produces each persona in two passes: (1) a **profile pass**, producing demographics, income band, goals, and risk profile consistent with the chosen archetype; (2) a **transaction synthesis pass**, generating 6–12 months of plausible day-by-day transactions (salary credits, category-tagged debits, EMI, occasional anomalies) that are statistically consistent with the profile and that deliberately embed one or more of the life-event signatures from Section 5.2, with the embedded ground-truth event stored separately so the detection engine’s output can be validated live, in front of the judge, against a known answer.

10.3 The six seed personas

Persona	Profile Sketch	Embedded Life Event(s)	Demo Purpose
Student	20, part-time stipend + parental transfers, minimal product holding	Approaching first-job transition	Shows acquisition-style guidance + onboarding bundle
Young Professional	26, first salaried job 8 months ago, growing surplus	Salary hike at month 6	Shows the canonical engagement workflow (Section 6.3) end-to-end
New Parent	33, dual income, recent large hospital debit	Child birth, nominee gap	Shows life-event detection sensitivity + insurance cross-sell
Future Home Buyer	35, rising savings rate, broker-merchant token payments	Home purchase intent	Shows long-horizon goal tracking + loan eligibility pre-check
Frequent Traveler	29, six trips in 90 days, no travel insurance	Frequent travel pattern	Shows time-sensitive, pre-trip engagement timing logic
Retiree	61, declining EMI burden, NPS step-up	Retirement-stage transition	Shows tone shift (advisory/preservation) vs. younger personas

10.4 What the generator outputs per persona

Transaction history (CSV/JSON, ledger-realistic with merchant category codes and geotags), spending habit summary, financial goals (mix of declared and to-be-inferred, to demonstrate both data paths into the Financial Life Graph), current SBI product holdings, and risk profile — i.e., everything the Financial Life Graph schema (Section 4.2) expects, so the rest of the pipeline runs unmodified against synthetic or (later) real data.

11. Judge Experience — Demo User Journey

The demo is designed as a single guided path so a judge with five minutes can experience the full product without narration gaps.

1. **Select a generated customer** — judge picks one of the six personas from the Customer Generator page; can optionally click “Generate New” to watch a fresh persona being synthesized live (proves the generator is real, not pre-baked).
 2. **View customer profile** — lands on the Customer Dashboard showing the Financial Life Graph summary at a glance.
 3. **View transaction history** — scrolls/filters the realistic transaction ledger, can spot the embedded life-event signal themselves before the AI flags it (builds anticipation).
 4. **Launch AI agent** — single button (“Run AI Engagement Cycle”) triggers the full LangGraph run for that customer.
 5. **Watch agent reasoning** — the Agent Activity Center streams each of the six agents’ outputs live, in the visual chain format described in Section 12.
 6. **Receive recommendations** — Recommendations Center populates with the ranked opportunity and its plain-language rationale.
 7. **Chat with the AI** — judge can ask the assistant questions about the recommendation (“why did you suggest this,” “what if my income drops next month”) and get graph-grounded answers, proving genuine reasoning rather than a canned script.
 8. **Execute mock actions** — judge clicks “Accept” on a recommendation, watches the Action Planning Agent generate steps, and “executes” the mock action in the Action Center, completing the loop and triggering the Follow-Up Agent’s scheduled check-in (shown as a future-dated card).
-

12. Dashboard Design — UX Specification

General design direction: **professional banking design, not an “AI demo” look.** SBI’s brand register (navy/blue with white space, restrained accent color, conservative typography weight) should anchor the palette over the generic dark-mode-purple-gradient aesthetic common in AI hackathon UIs. Use a single accent color (a saturated SBI-blue) for AI-attributed elements only, so the “this is the AI talking/acting” moments are visually distinct from static banking chrome without overwhelming the interface in gradients or glow effects.

12.1 Landing Page

- **Purpose:** Frame the product for a first-time judge/viewer in under 15 seconds.
- **Layout:** Centered hero with the product name/tagline, three-icon row showing the “detect → reason → engage” loop, single primary CTA (“Try the Demo”).
- **Components:** Hero text block, three-step visual strip, CTA button, a secondary “How it works” anchor link scrolling to a one-screen explainer.
- **Interactions:** Single click into the Customer Generator; no login wall for the demo path.

12.2 Customer Generator

- **Purpose:** Let the judge pick or generate a persona.
- **Layout:** A grid of six persona cards (Section 10.3), each with a one-line description and a small representative icon; a “Generate New Persona” button above the grid.
- **Components:** Persona cards (clickable), generation-in-progress skeleton state (showing the two-pass generation described in 10.2 as short status text, e.g. “Synthesizing transaction history...”), confirmation toast on selection.
- **Interactions:** Click a card → routes to Customer Dashboard for that persona; Generate New → streams a new card into the grid.

12.3 Customer Dashboard

- **Purpose:** At-a-glance summary of the selected customer’s Financial Life Graph.
- **Layout:** Left column — identity/demographics card and life-stage badge; center — key metric tiles (income trend, savings rate, idle-funds score, goal progress); right — quick links to Transaction History, Financial Life Graph View, and a prominent “Run AI Engagement Cycle” button.
- **Components:** Metric tiles with trend sparklines, life-stage badge, CTA button.
- **Interactions:** CTA routes to Agent Activity Center; tiles are clickable drill-ins to the relevant graph section.

12.4 Financial Life Graph View

- **Purpose:** Make the underlying data model (Section 4) tangible and inspectable, reinforcing that recommendations are graph-grounded.
- **Layout:** A node-link visualization (Section 4.1) — clusters for demographics, income, expenses, savings, investments, loans, insurance, goals — with edges drawn to recent events; a side panel showing the selected node’s value, confidence, and source evidence when clicked.
- **Components:** Interactive graph canvas, node detail side panel, a filter toggle to show only nodes updated in the last AI run.
- **Interactions:** Click a node → side panel shows value + evidence trail (e.g. which transaction caused this); hovering an edge shows the relationship label (e.g. “child birth → education goal”).

12.5 Agent Activity Center

- **Purpose:** The centerpiece “wow” screen — live visualization of the six agents reasoning in sequence.
- **Layout:** Vertical timeline, one card per agent in execution order, each card expanding to show its reasoning text as it completes (Section 12.6/13 detail the exact chain format).
- **Components:** Agent cards with status states (queued/running/complete), a live-updating chain diagram identical in spirit to the example in the brief’s “Agent Reasoning Visualization” section.
- **Interactions:** Cards auto-expand as each agent completes during a live run; judge can click any past card to re-expand and re-read.

12.6 AI Chat Interface

- **Purpose:** Conversational access to the assistant, grounded in the customer’s graph and the most recent agent run.
- **Layout:** Standard chat layout with a persistent context strip at top showing which customer/persona is active; inline “recommendation cards” can appear as chat messages, not just text.
- **Components:** Message stream, input box, inline recommendation cards, “why?” quick-action under every AI message that expands a short reasoning trace.
- **Interactions:** Free-text chat; “why?” expands reasoning; recommendation cards have Accept/Decline buttons inline.

12.7 Recommendations Center

- **Purpose:** A focused list view of all current ranked opportunities for the active customer.
- **Layout:** Ranked card list (highest fit_score first), each card showing product, one-line rationale, urgency badge, and Accept/Decline actions.

- **Components:** Recommendation cards, urgency badges (color-coded), filter by category (invest/protect/borrow/save).
- **Interactions:** Accept routes to Action Center with the plan pre-loaded; Decline triggers the Follow-Up Agent's snooze logic, visibly (a small confirmation: "We won't suggest this again for 60 days").

12.8 Action Center

- **Purpose:** Show the Action Planning Agent's step-by-step execution plan and let the judge complete the mock action.
- **Layout:** Stepper component (step 1...n), each step with a short description and a "complete" toggle; final step shows a confirmation state and the Follow-Up Agent's next scheduled check-in date.
- **Components:** Stepper, progress bar, confirmation state, follow-up-scheduled card.
- **Interactions:** Click through steps; final step completion animates into a "Follow-up scheduled for [date]" card, visibly closing the loop described in Section 6.3.

13. Agent Reasoning Visualization

13.1 Concept

This is the single most distinctive feature for differentiating SBI EngageAI from a chatbot demo: every recommendation is traceable, in the UI, as a literal chain of cause and effect, rendered live as the agents run. The brief’s own example is the canonical pattern we implement exactly:



13.2 Implementation

Each node in the chain corresponds 1:1 to a single agent’s output object from Section 6 (Agent 1’s “notable change,” Agent 2’s “event,” Agent 3’s “opportunity,” Agent 4’s “engagement plan,” etc.), so the visualization requires no separate authoring — it is a direct, faithful render of the LangGraph state object after each node executes, streamed to the frontend over Server-Sent Events as each agent completes. This guarantees the visualization can never drift from what the system actually did, which matters both for judge trust and for any future regulatory explainability requirement.

13.3 Visual treatment

Each step renders as a card with: a short title (the node’s headline conclusion, e.g. “Savings Capacity Increased”), a one-line evidence reference (e.g. “based on 2 salary credits, ₹62,000 → ₹74,000”), and a confidence indicator where applicable. Cards connect with a simple downward arrow/connector; completed cards stay visible and scrollable so the judge can review the full chain after the run finishes, not just watch it scroll past.

14. Technical Architecture

14.1 Stack

Layer	Technology	Notes
Frontend	React + TypeScript + Tailwind CSS	Component-driven; matches the eight pages in Section 12
Backend API	FastAPI (Python)	REST + Server-Sent Events for live agent streaming
Orchestration	LangGraph	Six-node agent graph per Section 6.4
LLM Provider	OpenAI or Gemini (provider-agnostic via a thin adapter)	Used for Agents 2, 3, 4 reasoning passes and the chat interface
Primary Database	PostgreSQL	Customer graph (JSONB), transactions, audit/events, persona seeds
Vector Database	Chroma (local) or Pinecone (hosted)	Embeddings of product catalog + past engagement rationale, used for semantic opportunity matching and chat retrieval
Cloud	AWS	EC2/ECS for API, RDS for Postgres, S3 for generated persona artifacts, optional Lambda for scheduled graph-refresh jobs

14.2 High-level architecture diagram (text form)



14.3 Component interactions

The frontend never calls the LLM or vector DB directly — all reasoning happens behind the FastAPI layer, which is the single point that constructs the LangGraph state, invokes agents in order, persists writes to Postgres (per the read/write rules in Section 6.5), and streams progress back over SSE. The vector DB is queried by Agent 3 (opportunity matching against the product catalog’s embedded descriptions) and by the Chat interface (retrieval-augmented answers grounded in the customer’s own graph history and prior rationale text, so the assistant can answer “why did you suggest this last month” accurately).

14.4 Core API surface

POST	/personas/generate	→ triggers two-pass synthetic generation
GET	/personas	→ list seed personas
GET	/customers/{id}/graph	→ current Financial Life Graph snapshot
GET	/customers/{id}/transactions	→ transaction ledger (paginated, filterable)
POST	/customers/{id}/run-agents	→ SSE stream: one event per agent completion
GET	/customers/{id}/recommendations	→ current ranked opportunity list
POST	/customers/{id}/recommendations/{rec_id}/accept	→ triggers Action Planning Agent
POST	/customers/{id}/recommendations/{rec_id}/decline	→ triggers Follow-Up snooze logic
GET	/customers/{id}/actions/{action_id}/steps	→ action plan stepper data

```
POST /customers/{id}/actions/{action_id}/steps/{n}/complete
POST /customers/{id}/chat → chat turn, RAG-grounded in graph + vector store
GET /customers/{id}/audit → full graph_events audit trail (powers Section 13)
```

14.5 Data flow (one engagement cycle)

1. Transaction stream (synthetic or real) → Agent 1
2. Agent 1 writes behavior deltas → customer_graph (Postgres) + graph_events (audit)
3. Agent 2 reads deltas + graph → writes life_event nodes → graph_events
4. Agent 3 reads graph (read-only) + product catalog embeddings (vector DB)
→ ranked opportunities (in-memory / cached, not graph-written)
5. Agent 4 reads opportunities + engagement history → engagement plan
→ persisted to recommendations table
6. Customer/judge action (accept/decline) → Agent 5 → action_steps table
7. Agent 6 → scheduled_followups table, time- or event-triggered re-entry to step 5
(Agent 4's queue)

14.6 Database schema (core tables)

```
customers(id, name, persona_type, created_at)
customer_graph(customer_id PK/FK, graph JSONB, confidence_overall, updated_at)
graph_events(id, customer_id, agent_name, field_path, old_value, new_value,
             evidence_refs JSONB, created_at)
transactions(id, customer_id, date, amount, direction, merchant_category,
             merchant_name, geotag, narration)
personas(id, archetype, profile JSONB, embedded_events JSONB)
recommendations(id, customer_id, product, fit_score, rationale, urgency,
               status, created_at)
action_steps(id, recommendation_id, step_number, description, status)
scheduled_followups(id, recommendation_id, due_at, status)
product_catalog(id, name, category, eligibility_rules JSONB, description,
               embedding_id)
```

This schema directly mirrors the Financial Life Graph logical model (Section 4.2) and the six-agent I/O contracts (Section 6.2), so an implementer can generate the FastAPI Pydantic models and SQLAlchemy ORM classes straight from this section with no further design step required.

15. Hackathon MVP Scope

15.1 Must Have (build first, demo-blocking)

- Synthetic Customer Generator producing the six seed personas with embedded life events (Section 10).
- Financial Life Graph schema implemented in Postgres with the audit (`graph_events`) table.
- All six agents implemented as LangGraph nodes with real LLM reasoning calls (not hard-coded text) for at least the canonical Young Professional → salary-hike → SIP-recommendation workflow (Section 6.3).
- Agent Activity Center UI streaming live agent output via SSE (Section 13).
- Recommendations Center + Action Center with at least one fully clickable accept → step-through → complete path.
- Chat interface grounded in the active customer's graph (basic RAG, can be a simplified single-pass retrieval if time-constrained).

15.2 Should Have (strong differentiation if time allows)

- Financial Life Graph node-link visualization (Section 12.4) — high visual impact, moderate build cost.
- All 20 engagement scenarios from Section 7 wired as scoreable data rather than just the one demoed end-to-end.
- Follow-Up Agent's scheduled re-engagement visibly demonstrated (e.g. a “simulate 30 days forward” button that re-triggers Agent 6 logic).
- Public/acquisition-mode chat entry point (Section 9), even as a second, simplified route reusing the same backend.

15.3 Nice to Have (only if ahead of schedule)

- Multiple personas fully wired end-to-end (beyond the one canonical demo path) so judges can pick any of the six and get a real, distinct run.
- Voice input on the chat interface.
- A lightweight admin view showing the product catalog and engagement-scenario table as editable data, proving the “data not hard-coded logic” claim from Section 7 live.

15.4 What is realistic in 24-48 hours

A two-person-or-larger team can realistically deliver the full Must Have list plus most of the Should Have list within 24-48 hours, because the architecture is intentionally narrow at the data layer (one Postgres schema, one vector store) and the heaviest “intelligence” work is prompt design for Agents 2-4 rather than custom ML training — there is no model-training critical path. The recommended build order is: (1) schema + persona generator, (2) Agent 1 + 2 with real reasoning over one persona's data, (3) Agent Activity Center streaming UI against that single working pipeline (this is the screen judges remember — prioritize it early), (4) Agents 3-6 plus Recommendations/Action Center, (5) chat interface, (6) extend to remaining personas and scenarios as time allows.

16. Business Impact & KPIs

KPI	Definition	Why It Matters Here
Customer Engagement Rate	% of customers who interact with a proactive AI touchpoint within 7 days of it being generated	Direct measure of the “Digital Engagement” theme objective
Product Adoption Rate	% of accepted recommendations that convert to an actual product action (SIP top-up, policy purchase, etc.)	Proves engagement converts, satisfying the Digital Adoption secondary theme without a separate funnel
Retention Rate	% of engaged customers still actively using SBI digital channels at 90/180 days	Engagement-driven retention is the core long-term value claim
Customer Lifetime Value (CLV) uplift	Incremental CLV for customers in the AI-engaged cohort vs. a control cohort	The metric that ultimately justifies the business case to SBI leadership
Conversion Rate (acquisition surface)	% of public-chat prospects who open an SBI account within 30 days of a guidance session	Validates the Customer Acquisition emergence claim in Section 9
Follow-Up Completion Rate	% of scheduled follow-ups (Agent 6) that result in either completion or an explicit decline, vs. silent drop-off	Measures whether the loop actually closes, not just opens

16.1 How SBI benefits

Beyond the direct KPI uplift, the system reduces dependency on branch/RM bandwidth for routine cross-sell conversations (the AI does the noticing and the first-pass explaining; human RMs are freed for high-value, high-trust conversations), produces a continuously improving signal of what actually drives adoption (since every engagement, accept, and decline is logged with its full reasoning chain — a dataset SBI does not currently have in this structured a form), and creates a defensible, hard-to-copy data asset: the longer a customer’s Financial Life Graph runs, the better the system’s predictions and the harder it becomes for a competitor app to replicate the same quality of personalization from a cold start.

17. Commercial Potential

17.1 Revenue impact

Cross-sell conversion driven by reasoned, life-event-timed recommendations typically outperforms generic segment-based campaigns by a wide margin in retail banking, because timing and relevance — not just targeting — drive conversion; even a conservative few-percentage-point lift in cross-sell conversion across SBI's retail base translates to meaningful incremental fee and interest income given the scale of the customer base, without requiring new product manufacturing — the system monetizes products SBI already has.

17.2 Cross-selling impact

Because the Opportunity Discovery Agent scores across SBI's full product catalog (investment, insurance, credit, and loan products) against one unified customer graph, the natural tendency is toward **multi-product relationships per customer** rather than single-product engagement, which is the single strongest lever on long-term CLV in retail banking.

17.3 Customer retention impact

Proactive, relevant engagement is a well-established retention lever because it shifts the customer relationship from transactional to advisory — a customer who feels understood is less likely to consider switching primary banking relationships, particularly against fintech competitors whose main advantage today is exactly this kind of personalized, proactive UX.

17.4 Operational cost reduction

Automating the “noticing and first explanation” stage of relationship management reduces the volume of low-value inbound queries to call centers and branches (customers calling to ask “what should I do with my bonus” can instead get a reasoned first answer from the assistant), and the Synthetic Customer Generator itself has reuse value for SBI's internal model testing, QA, and staff-training environments, reducing the cost of building those environments separately.

18. Round 1 Submission Content

Project Title: SBI EngageAI — The Autonomous Financial Companion

Brief Description: SBI EngageAI is an autonomous, multi-agent AI layer that continuously analyzes customer financial behavior, detects life events as they happen, and proactively engages customers with reasoned, relevant recommendations — turning every customer interaction from a request-response transaction into an ongoing, advisory relationship.

Proposed Solution: A six-agent system (Financial Behavior Analyzer, Life Event Detection, Opportunity Discovery, Engagement Strategy, Action Planning, and Follow-Up agents) operates on a continuously evolving Financial Life Graph per customer. The system detects events such as salary hikes, marriage, child birth, relocation, and home-purchase intent from transaction patterns, reasons about the resulting financial opportunity, and proactively delivers a recommendation through the right channel and tone — with every step of that reasoning visible and explainable to the customer. Demonstrated on a Synthetic Customer Generator producing six realistic personas with embedded, ground-truth life events.

Business Model / Commercial Potential: Revenue is generated indirectly through increased cross-sell conversion and CLV uplift on SBI’s existing product catalog (no new product manufacturing required), with secondary value from reduced call-center/branch load for routine advisory queries and reuse of the Synthetic Customer Generator for internal QA and staff training. See Sections 16–17 for full KPI and impact modeling.

Technology Stack: React + TypeScript + Tailwind (frontend), FastAPI (backend), PostgreSQL (primary store), LangGraph (agent orchestration), OpenAI/Gemini (LLM reasoning), Chroma/Pinecone (vector retrieval), AWS (cloud deployment). Full detail in Section 14.

Process Flow: Transaction stream → Behavior Analyzer → Life Event Detection → Opportunity Discovery → Engagement Strategy → customer touchpoint → Action Planning (on acceptance) → Follow-Up Agent (closes the loop, re-enters the cycle). See Section 6.3 for the fully worked example.

Architecture Description: A FastAPI backend orchestrates a six-node LangGraph agent graph against a PostgreSQL-backed Financial Life Graph, with a vector database for semantic product matching and chat retrieval; the React frontend streams live agent reasoning via Server-Sent Events into a dedicated Agent Activity Center, making the system’s reasoning auditable and demo-able in real time. Full diagram and schema in Section 14.

19. Pitch Deck — 12 Slides

- Slide 1 — Title** Content: “SBI EngageAI — Your Financial Copilot.” Team name, hackathon track (Digital Engagement). Visual: Clean SBI-blue title card, the three-step “detect → reason → engage” icon strip. Speaker notes: Open with the inversion line — “Banking today waits for you to ask. We built the layer that notices first.”
- Slide 2 — The Problem** Content: Customers don’t proactively manage their financial life; banks don’t proactively help them, despite having the data to. Visual: Simple before/after split — “Customer asks → Bank responds” vs. “AI detects → AI engages.” Speaker notes: Ground this in the actual problem statement language to show direct alignment.
- Slide 3 — Our Insight** Content: Engagement, adoption, and acquisition are not three separate problems — they are three outputs of one reasoning engine running on one customer graph. Visual: One circle (the engine) with three labeled outputs branching from it. Speaker notes: This is the “we understood the brief at a structural level” slide — emphasize it’s a deliberate design choice, not a simplification.
- Slide 4 — The Product** Content: Define SBI EngageAI as an AI layer, not an app or a chatbot — six autonomous agents, one Financial Life Graph per customer. Visual: The six-agent pipeline diagram (Section 6.3), simplified to one row of icons. Speaker notes: “ChatGPT + Financial Advisor + Autonomous Banking Agent” — use this line verbatim, it’s memorable.
- Slide 5 — Life Event Detection** Content: How the system reads transaction patterns into named life events with confidence scores. Visual: One worked example from Section 5.2’s table (e.g. Salary Hike), shown as signal → reasoning → event. Speaker notes: Pick the most visually clean example (salary hike) for the live audience version even if the demo itself uses a richer one.
- Slide 6 — The Six Agents** Content: One line each for the six agents and their role in the loop. Visual: Vertical pipeline matching the Agent Activity Center UI, so the slide and the live demo visually rhyme. Speaker notes: Tell judges “you’re about to see this exact pipeline run live.”
- Slide 7 — Live Demo Transition** Content: “Let’s watch it reason.” (Minimal text slide, transition cue.) Visual: Just the product logo/wordmark. Speaker notes: Switch to the live application here; see Section 20 for the demo script.
- Slide 8 — Engagement, Adoption, Acquisition (Post-Demo)** Content: Recap that what was just shown is simultaneously an engagement event, an adoption funnel, and (with a flipped data source) an acquisition surface. Visual: The same one-engine/three-outputs diagram from Slide 3, now annotated with what was just demoed. Speaker notes: Close the loop back to the structural insight from Slide 3 — judges should feel the architecture argument was proven, not just claimed.
- Slide 9 — Architecture** Content: Condensed version of the Section 14.2 diagram. Visual: The architecture diagram itself, simplified to 5–6 boxes. Speaker notes: Emphasize production-readiness: real stack choices, not hackathon shortcuts, with a clear path from this MVP to a production rollout.
- Slide 10 — Business Impact** Content: The five KPIs from Section 16, with the revenue/retention logic from Section 17 condensed to two sentences. Visual: A simple KPI strip (5 labeled metrics, no fabricated numbers — frame as the measurement plan, not invented results). Speaker notes: Be explicit that these are the KPIs the system is *instrumented* to produce, not simulated results, to maintain credibility with judges.
- Slide 11 — Why This Wins** Content: The four differentiators from Section 21 (vs. chatbots, recommendation engines, rule-based systems, generic AI assistants). Visual: A 2x2 or simple comparison row. Speaker notes: This is the closing argument slide — keep it tight, one line per differentiator.
- Slide 12 — Ask / Close** Content: What the team is asking for (advancement to next round) and the one-line vision restated. Visual: Title-card bookend matching Slide 1. Speaker notes: End on the inversion line from Slide 1 again — “We didn’t build a better way to answer questions. We built the layer that notices first.”
-

20. Demo Script (3 Minutes)

0:00-0:25 — Setup “This is SBI EngageAI. We’re not showing you a chatbot — we’re showing you an AI that already noticed something about this customer before we even opened the app.” Select the **Young Professional** persona from the Customer Generator.

0:25-0:50 — Customer Context Land on the Customer Dashboard. Point at the income trend tile. “Eight months into her first job, here’s her transaction history” — quick scroll through the ledger, pause on the visibly larger recent salary credits without naming the event yet, building anticipation.

0:50-1:30 — Launch & Life Event Detection Click “Run AI Engagement Cycle.” As the Agent Activity Center streams: narrate Agent 1’s behavior summary, then Agent 2’s output landing — “Salary Hike — confidence 0.88” — appearing live. “The system just did, in seconds, what would normally need a relationship manager to notice from a printed statement.”

1:30-2:05 — Reasoning Chain & Recommendation Let the full reasoning chain render (Section 13’s exact pattern: income increased → savings capacity increased → opportunity identified → engagement strategy generated → recommendation delivered). Click into the Recommendations Center to show the SIP top-up card with its rationale.

2:05-2:35 — Chat & Action Open chat, ask the assistant live: “Why are you suggesting this instead of a fixed deposit?” — let it answer using graph-grounded reasoning. Click Accept on the recommendation, step through the Action Center stepper to completion.

2:35-3:00 — Close the Loop Show the Follow-Up Agent’s scheduled check-in card appearing. “This isn’t a one-time popup — thirty days from now, the same agent checks back in, the way a relationship manager would.” End on the line: “Detect, reason, engage, follow up — fully autonomous, fully explainable.”

21. Differentiators

Compared To	Their Limitation	SBI EngageAI's Answer
Traditional chatbots	Reactive — wait for the customer to ask, no memory of financial context across sessions	Proactive by design; persistent Financial Life Graph carries context across every interaction
Banking recommendation engines	Score products against a static customer segment, not a live behavioral trajectory	Opportunity Discovery Agent scores against continuously-updated graph state, triggered by detected events, not batch segmentation
Rule-based systems	Brittle, hand-coded triggers (“if balance > X, show Y”) that can’t weigh corroborating-but-imperfect signals	LLM-reasoned event detection with confidence scoring (Section 5.3) handles ambiguous, multi-signal evidence the way a human advisor would
Generic AI assistants	Single-prompt “do everything” pattern with no inspectable reasoning, no persistence, no action execution	Six specialized agents with explicit inputs/outputs, a fully auditable reasoning chain (Section 13), and a real action-to-follow-up loop (Section 6.3)

The throughline across all four comparisons is **true agentic behavior**: the system does not just answer or recommend — it perceives, reasons, acts, and revisits, autonomously and continuously, with every step traceable back to evidence in the customer’s own data.

22. Build Checklist for an Autonomous Coding Agent

This section is addressed directly to an AI coding agent (e.g. Antigravity AI) tasked with implementing this specification. Build in this order; each step references the section of this document containing its full detail.

1. **Scaffold repo:** `frontend/` (React + TS + Tailwind), `backend/` (FastAPI), `agents/` (LangGraph graph definition), `infra/` (AWS IaC, optional for hackathon scope).
2. **Database:** Implement the schema in Section 14.6 against PostgreSQL; write migrations.
3. **Synthetic Customer Generator:** Implement the two-pass LLM generation described in Section 10.2, seeding the six personas from Section 10.3, including embedded ground-truth life events stored in `personas.embedded_events`.
4. **Financial Life Graph:** Implement the schema from Section 4.2 as the `customer_graph.graph` JSONB document; implement the `graph_events` audit-writer used by every writing agent.
5. **Agent graph:** Implement the six LangGraph nodes per the I/O contracts in Section 6.2, respecting the read/write separation rules in Section 6.5; wire the conditional edges (e.g. confidence threshold gating from Agent 2 to Agent 3, per Section 5.3).
6. **API layer:** Implement the endpoints listed in Section 14.4, with `/customers/{id}/run-agents` streaming agent completion events over SSE for the Agent Activity Center.
7. **Frontend pages:** Build the eight pages specified in Section 12, in priority order Landing → Customer Generator → Customer Dashboard → Agent Activity Center → Recommendations Center → Action Center → Chat → Financial Life Graph View, matching the Must/Should/Nice priority in Section 15.
8. **Reasoning visualization:** Implement the live chain renderer described in Section 13, consuming the SSE stream directly — do not hand-author the chain text; render it from agent output objects.
9. **Vector store:** Embed the product catalog (Section 14.4/14.6 `product_catalog` table) into Chroma/Pinecone for Agent 3’s semantic matching and for chat retrieval.
10. **Validate against the canonical workflow:** Confirm the Young Professional persona produces the exact Section 6.3 chain (salary hike → SIP top-up) end-to-end before extending to other personas.
11. **Extend to remaining personas and the engagement scenario library** (Section 7) as time allows, per the Should/Nice Have scope in Section 15.2–15.3.
12. **Prepare submission artifacts:** Section 18 copy for Round 1 text fields, Section 19 deck content for slide generation, Section 20 script for the recorded/live demo walkthrough.

— End of document —